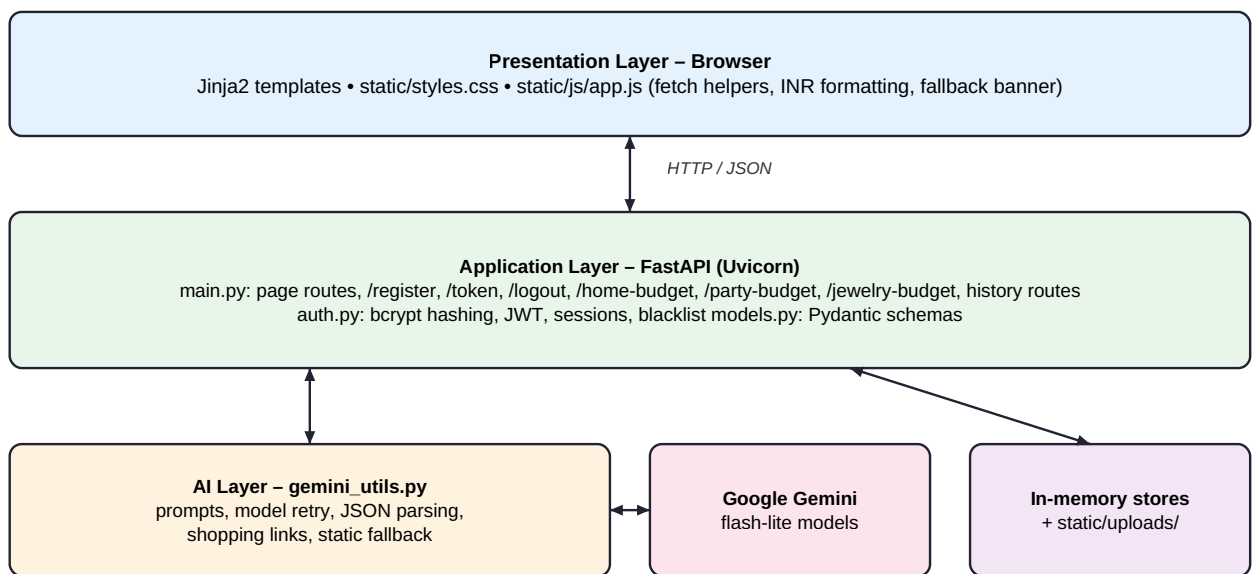


Solution Architecture

Date	04 October 2026
Team ID	POCKETSMART-TEAM
Project Name	PocketSmart AI – Smart Budget & Recommendation Assistant using Gemini Models
Maximum Marks	3 Marks

Architecture Overview



Module Responsibilities

Layer	Main Modules	Responsibility	Owner
Frontend	templates/ (index, login, register, dashboard, home_planner, party_planner, jewelry_planner, history), static/styles.css, static/js/app.js	Forms, display, styling, API calls and INR formatting	Frontend Lead
API / Routing	main.py	Page and API routes, dependency-based authentication, session cleanup task	Backend Lead
Authentication	auth.py	Password hashing, JWT creation/verification, sessions and token blacklist	Backend Lead

AI	gemini_utils.py	Gemini prompts, model fallback and retry, JSON extraction, shopping links, static fallback, upload handling, history	AI Integration Lead
Schemas / Validation	models.py	Pydantic request models and validators	Testing & Documentation Lead + Backend Lead
Testing	tests/test_app.py	API tests with Gemini mocked	Testing & Documentation Lead

Data Flow

The user signs in and receives a token. A planner form is posted to the matching endpoint, where Pydantic validates it. The AI layer asks Gemini for JSON, tries the next model or falls back to a static estimate on failure, adds shopping links, and returns the result. The result is stored in the user's history and rendered by the browser.